Resolve.

DOCUMENTATION LIBRARY / 02

Implementation guide

Workflow services, state and integrations

A source-level guide to reconciliation, guarded refunds, evidence investigations, aftercare and the server-side SDK.

COMPILATION DATE

12 November 2026

Source chapters retain their recorded dates and validation scope. Local, simulated and account-backed evidence are distinguished. This volume does not report a new production acceptance run.

Implementation map

This module inventory is read from Python source without importing the application or accessing configured services. It is an implementation map, not runtime test coverage.

Module	Declared responsibility / main entry points
aftercare.py	Order-bound customer support for explicitly enabled server integrations.
aftercare_routes.py	A bounded app-to-Resolve aftercare contract, separate from merchant administration.
ai_extraction.py	Propose merchant promises from a chat transcript; never records or approves them.
ai_privacy.py	Mask personal payment details before an LLM call, and flag suspicious buyer text.
ai_routes.py	Merchant API for AI promise proposals. Proposals are never saved as promises.
ai_usage.py	In-process cache and per-merchant daily token usage for promise extraction.
approvals.py	Persist exact merchant approval decisions; this module never executes refunds.
assurance_routes.py	Assurance APIs share existing ownership, role, body and buyer capability boundaries.
auth.py	Entry points: get_store, get_current_merchant, get_commerce_merchant
case_operations.py	Merchant triage fields and derived workflow state; no payment mutations.
case_workspace.py	Case lifecycle, immutable attachments, and a merchant-owned exception queue.
commerce_compare.py	Compare an app snapshot only with facts from an applicable PayPal check.
commerce_models.py	Allowlisted, merchant-reported order evidence and PayPal comparison contracts.
commerce_routes.py	Entry points: PrivateCommerceRoute, sync_commerce_order, register_commerce_order, list_commerce_orders, get_registered_order, open_commerce_case, get_commerce_order
commerce_workspace.py	Merchant order catalog and its optional link to a resolution case.
customer_account.py	Mailbox verification and case access, separate from merchant sessions.
customer_email.py	Transactional delivery queue for optional customer emails.
customer_evidence.py	Bounded, downloadable case files submitted by the customer.
customer_google.py	One-time Google OIDC for the buyer portal; reuses mailbox-scoped sessions.
customer_intake.py	One-use order invites. Redeeming an invite never looks up another case.
customer_portal.py	Case-scoped customer access and a separate, explicitly shared conversation.
customer_progress.py	Curated customer milestones; internal notes and raw provider data stay private.
customer_routes.py	Entry points: PrivatePortalRoute, merchant_inbox, customer_link_status, read_helpdesk, link_helpdesk, issue_customer_link, revoke_customer_link
customer_smtp.py	Run the optional Nodemailer transport without putting secrets in arguments.
dispute_evidence.py	Reason-specific document requirements; a present document is not a winning dispute.
dispute_guard.py	Versioned capture-wide dispute holds. Recorded provider names are not provider verification.
errors.py	Entry points: StoreError
evidence_models.py	Private merchant contracts for the Evidence SDK.

Module	Declared responsibility / main entry points
evidence_paypal.py	Collect seller-scoped read-only PayPal facts for SDK evidence.
evidence_workspace.py	Merchant-scoped idempotent evidence, kept separate from execution state.
executions.py	Durable first full-capture sandbox refunds; network calls never hold a DB lock.
followup_worker.py	Background reads and customer notifications; never submits a refund.
helpdesk.py	Read and attach a Zendesk ticket owned by the configured helpdesk.
inference.py	Bounded, non-streaming JSON adapters; fixed origins, no tools or fallback routes.
investigation_models.py	Advisory investigation contracts. No execution instruction is representable.
investigation_refresh.py	One server-authorized linked Sandbox resource read, never model-supplied URLs.
investigation_routes.py	Full merchant authentication only; advisory review has no payment side effects.
investigation_worker.py	Separate bounded investigation worker; never starts the execution worker.
investigations.py	Private, revision-bound advisory investigations using the existing worker/store pattern.
main.py	Entry points: health, readiness, create_case, list_cases, get_case, update_commitment, get_timeline
manage.py	Trusted local administrator commands. Never expose these through HTTP.
merchant_auth.py	Durable merchant browser sessions and optional PayPal identity login.
merchant_auth_routes.py	Private responses for the merchant browser login and PayPal callback.
migrate_sqlite.py	Copy a read-only snapshot of a Resolve SQLite database into an empty Postgres schema.
models.py	Entry points: PaymentReference, RefundCommitment, CaseCreate, CaseResponse, CaseAction, CaseAttachment, ReviewException
next_action.py	What is missing before a refund, and a short explanation of the next action.
operations_worker.py	Process signed support imports and daily promise alerts; never executes money or sends email.
paypal.py	First-party sandbox payment APIs. Hosts are fixed; links are never followed.
paypal_history.py	Order-scoped Sandbox history and a durable inbox for authenticated events.
platform_connections.py	Encrypted workspace credentials and explicit provider checks.
platform_identity.py	Individual workspace identities; existing merchant keys remain administrator credentials.
platform_imports.py	Authenticated Zendesk reads, source-preserving imports and durable webhook jobs.
platform_models.py	Reusable response contracts for merchant operations and buyer decisions.
platform_operations.py	Version-bound buyer decisions, guided returns, portfolio alerts and evidence packages.
platform_routes.py	Merchant operations API and capability-scoped buyer actions.
postgres.py	PostgreSQL transaction driver for the shared Resolve repository.
promise_health.py	Broken-promise detector: compares accepted refund promises with refunds already on the payment.
promise_health_routes.py	Merchant API for the broken-promise detector (read-only).

Module	Declared responsibility / main entry points
reconciliation.py	Deterministic calculations over local development records; no payment calls.
remedies.py	Suggested remedies: code-built options from the reconciliation and the return policy, an
remedies_routes.py	Merchant API for suggested remedies. Suggestions are code-built; creating promises needs the
resolution_assurance.py	Customer-safe refund journeys, shared context and recoverable issue receipts.
resolution_metrics.py	Time to verified resolution, read from a case timeline.
resolution_models.py	Public contracts for the shared remedy workspace.
resolution_providers.py	Read-only evidence adapters. No provider data is mutated here.
resolution_routes.py	Entry points: create_resolution, list_resolutions, upsert_evidence_case, get_resolution, link_evidence_record, submit_merchant_evidence, refresh_case_evidence
resolution_state.py	Derive promise status and the refund approval gate from recorded evidence.
resolution_workspace.py	Transactional promises, evidence and read-only provider reconciliation.
return_assurance.py	Explainable return checks, inspection findings and authenticated reusable labels.
sdk_events.py	Reconciled safe snapshots for resumable app synchronization, independent of webhook delivery.
settings.py	Entry points: Settings
storage.py	Resolve repository and its local SQLite transaction implementation.
synthetic_investigations.py	Fixed labeled synthetic records and loopback HTTP inference stand-ins. No real data.
worker.py	Trusted local recovery of already-approved sandbox executions; no new consent.
workspace_routes.py	Entry points: bounded_body, receive_paypal_webhook, refresh_paypal_history, close_case, reopen_case, upload_attachment, list_attachments

Reading map

This volume brings together the maintained source documents listed below. Source chapters retain their recorded evidence dates and assumptions. Later corrections in the engineering log take precedence over older snapshots.

Chapter	Source in the project
Resolution workspace	docs/RESOLUTION_WORKSPACE.md
Backend operations	docs/BACKEND_OPERATIONS.md
PayPal reconciliation	docs/PAYPAL_RECONCILIATION.md
Refund execution	docs/REFUND_EXECUTION.md
Aftercare reliability	docs/AFTERCARE_RELIABILITY.md
Embedded aftercare	docs/SDK_AFTERCARE.md
Automatic investigations	docs/AI_INVESTIGATIONS.md
Server-side SDK	sdk/typescript/README.md
Checkout integration	examples/checkout-store/README.md

Resolution workspace

Source: docs/RESOLUTION_WORKSPACE.md

Resolution workspace

The merchant API tracks several promises and their evidence in one case, even when there is no captured payment. It can link one existing refund case; that case continues to own the refund ledger, approvals and execution.

Route	Contract
`POST /resolution-cases`	Create an owned workspace, optionally linked to a refund case.
`GET /resolution-cases`, `GET /resolution-cases/{id}`	List cases and inspect current promise and evidence state.
`POST /resolution-cases/{id}/promises`	Record a promise with source, due date, typed remedy and optional prerequisite promises.
`PUT /resolution-cases/{id}/promises/{promise_id}`	Replace a promise with an explicit expected revision; retain the old promise and its evidence in the timeline.
`POST /resolution-cases/{id}/promises/{promise_id}/evidence`	Record merchant evidence for an exact revision. Only an explicit internal warehouse receipt or inspection attestation can satisfy a manual return milestone.
`POST /resolution-cases/{id}/promises/{promise_id}/refresh`	Read provider evidence for the current revision. May update local evidence and consent; never issues a refund or changes provider data.
`GET /resolution-cases/{id}/timeline`	Paginated, ordered history of promises, merchant evidence and provider observations.
`GET /resolution-cases/{id}/evidence-bundle`	Private export for a merchant's review or dispute handoff. No evidence is submitted to PayPal.

Each request uses the existing merchant bearer token. Prerequisites reference earlier promises in the same workspace. The workspace tracks one cumulative refund promise for a linked refund case. The amount must agree with the accepted commitment on that case; its existing approval and execution routes remain the only way to submit money movement. A prerequisite with incomplete evidence blocks that refund. A changed revision cannot inherit the old revision's proof.

The outcome and its source are separate: a merchant's inspection attestation can satisfy an internal condition, but cannot serve as PayPal or carrier proof. An order cancellation and a payment release are two independent facts. A formal dispute status does not close other promises. External IDs are linked by the merchant and independently checked in the configured provider account; Resolve does not infer cross-system identity from similar amounts or names.

Try a conditional refund

Run `uv run uvicorn app.main:app --reload` from `backend` and open Swagger (http://127.0.0.1:8000/docs). Authorize with the private Resolve token.

1. Create the ordinary refund case with its payment IDs and accepted commitment.

Create a resolution case with that ID as `refund_case_id`. For shipment-only and subscription cases, omit it; use `order_reference` for shipment cases.

2. Create a return promise in the resolution case:

```
{
  "promise": "Confirm warehouse receipt before sending the refund",
  "source_note": "Recorded from the agreed support resolution",
  "target": {
    "kind": "return",
    "milestone": "warehouse_received",
```

```
      "return_reference": "RMA-DEMO-1"
    }
  }
```

3. Create a second promise with target `{"kind":"refund","amount":"30.00"}`

and the return promise ID in `requires`. This assumes a $30 accepted commitment on the linked refund case. The refund case's reconciliation reports `awaiting_prerequisite` until the condition is satisfied.

4. POST evidence with `{"expected_revision":1,"finding":"confirmed","note":"Warehouse logged receipt"}`.

This confirms only the internal return milestone, under the merchant's identity. Refresh the refund promise with `{"expected_revision":1}` to discover PayPal refund history. Then use the existing approval and execution routes if another refund is actually owed.

5. View the case and timeline. The timeline response contains separate

`items` and `refund_items` arrays with their own totals, paginated with the same requested offset and limit. The evidence bundle includes attachment metadata for the linked refund case; use its existing attachment routes for file bytes. Only the case owner can access it.

Remedy details and configuration

target.kind	Evidence Resolve checks
`refund`	Existing PayPal Sandbox order, capture and refund records; amounts in the workspace are based on provider observations, not manual ledger rows. History freshness and verified remaining capture balance are separate from how much of the promise is still owed.
`return`	An EasyPost tracker for `carrier_delivered`; a revision-specific merchant finding for `warehouse_received` or `inspection_passed`. Carrier evidence on an internal milestone is supplementary.
`replacement`	A replacement reference and its linked EasyPost tracker; the expected carrier and tracking code must match. A shipping label is pending, not shipped.
`order_cancellation`	Shopify's `cancelledAt` and/or PayPal authorization status `VOIDED`. If both references are given, both must confirm. A captured payment needs a separate refund promise; it is not shown as an authorization release.
`subscription`	PayPal status `CANCELLED` or `SUSPENDED` for the requested action.
`store_credit_or_exchange`	Exact Shopify credit transaction, account, amount, and currency; or the specified exchange line item in a matching Shopify fulfillment with shipment movement. Credit is historical issuance, not a current spendable-balance check.
`billing_correction`	PayPal invoice cancellation; or capture evidence plus a required, separately verified refund promise for a reported duplicate or incorrect charge. Two capture records alone cannot prove the charges were duplicates.
`dispute_handoff`	Formal PayPal dispute status and seller response deadline when supplied, with a matching seller-side transaction. The private JSON evidence bundle can be reviewed and shared by the merchant. Resolve does not send it or act on the formal dispute.

PayPal evidence uses the existing Sandbox configuration and seller binding. Shopify and EasyPost are optional. Set `RESOLVE_RESOLUTION_CONNECTOR_MERCHANT_ID` to the Resolve merchant UUID that owns the connected accounts. For Shopify, set `RESOLVE_SHOPIFY_SHOP_DOMAIN` to its `*.myshopify.com` hostname and `RESOLVE_SHOPIFY_ADMIN_TOKEN` to the shop's server-side Admin token with permission to read the relevant orders and store-credit transactions. The connector queries Admin GraphQL `2026-10`. For EasyPost, set `RESOLVE_EASYPOST_API_KEY` and use a tracker ID already present in that account. The response labels EasyPost test observations when the API reports test mode. Keep these values in the ignored local `.env`.

Connectors perform on-demand evidence refreshes. Provider proof is an observation as of its recorded time. Non-refund connectors do not subscribe to webhooks or perform cancellations, exchanges, shipping purchases, or credits. The existing refund worker and webhook processing continue to update the linked refund case. No connector was tested against a live merchant account in this change.

Provider references: PayPal authorizations (https://developer.paypal.com/api/payments/v2/authorizations-get/), subscriptions (https://developer.paypal.com/api/subscriptions/v1/subscriptions-get/), invoices (https://developer.paypal.com/api/invoicing/v2/), disputes (https://developer.paypal.com/api/customer-disputes/v1/disputes-get/), Shopify order (https://shopify.dev/docs/api/admin-graphql/latest/objects/Order), credit transaction (https://shopify.dev/docs/api/admin-graphql/latest/objects/StoreCreditAccountCreditTransaction), fulfillment (https://shopify.dev/docs/api/admin-graphql/latest/objects/Fulfillment), and EasyPost tracker (https://docs.easypost.com/docs/trackers).

Backend operations

Source: docs/BACKEND_OPERATIONS.md

Backend operations and case workspace

Implemented API contract

Route	Behavior
`GET /cases?limit=20&offset=0&capture_id=...`	Merchant-owned case page, newest inserted rows first; optional exact capture filter. Limit 1-100, offset 0-100000.
`PUT /cases/{id}/commitment`	`{state_fingerprint, commitment}`. Validates the full commitment against the capture and current reconciliation fingerprint, saves revision evidence, and invalidates old approvals.
`GET /cases/{id}/timeline?limit=50&offset=0`	Recorded case/payment history for the owning merchant's shared capture, including original source case IDs.

Authentication is the existing Resolve merchant token. Case/payment queries never include another merchant's data. A missing/foreign case is `404`; stale changes or an active execution are `409`; invalid request/amount/pagination is `422`. Submitting the already-current commitment is an idempotent read/no-op. Different updates require the current fingerprint from reconciliation.

Changing a commitment condition starts a new evidence epoch: previous condition checks remain in history but do not satisfy the revised condition. No new physical evidence or human action is fabricated. A new check is required. Captured amount, currency and case ownership are not edited through this route.

Timeline entries retain source, actor account/system/unknown attribution, source case and actual recorded UTC timestamp. New case/manual-refund/commitment changes have durable events. Existing timestamped approvals, execution events, condition checks and provider observations are included. Historical case creation/manual refund changes without timestamps are not reconstructed. Ordering is UTC followed by a deterministic event ID, not a claim of a global external event order.

Local recovery worker

Run one bounded recovery pass from `backend`:

```
uv run python -m app.worker --once
```

For a long-running local worker:

```
uv run python -m app.worker --loop --interval 30 --limit 20
```

The trusted local worker uses only the configured merchant's already-reserved executions. It cannot create commitments or approvals. Pending known refunds are read/refreshed. Reserved/unknown or expired submission leases may be recovered with the saved provider identity/body, within the supported POST window and only when the refund flag is enabled. Short backoff prevents repeated immediate work; active leases, identity mismatches and expired recovery windows are respected. Writes-disabled mode can still refresh known pending refunds when reads are enabled.

Output contains counts and opaque record IDs/status codes, never credentials, tokens, customer conversations or raw provider errors. A process stop does not erase reservations. Multiple workers rely on the same database lease rules.

A failed first full-refund execution that leaves a commitment outstanding reports `execution_failed` / `review_failed_execution`, rather than recommending another approval for an already-claimed capture. Later verified refunds can still fulfill the commitment; failed execution history does not erase money already returned.

Response pages are bounded. Timeline assembly currently reads the capture's recorded history into memory before slicing; a deployed high-volume version needs database-side pagination/aggregation. Historical rows are not assigned invented creation times or actor identities.

This is a local process, not installed as an operating-system service or a hosted scheduler. The same worker processes verified Sandbox webhook deliveries; see registration and processing (PAYPAL_RECONCILIATION.md). A hosted worker remains future work.

PayPal reconciliation

Source: docs/PAYPAL_RECONCILIATION.md

PayPal Sandbox reconciliation and case workflow

Refresh history and refund the remaining amount

1. Use a case with its actual PayPal Sandbox order and capture IDs. The order must have one purchase unit and one capture for the configured Sandbox seller, in USD.

2. Call `POST /cases/{case_id}/paypal-refund-history/refresh` with the merchant bearer key. This is a read of PayPal. It discovers refunds in the order, verifies every refund by ID and parent capture, compares completed totals against the cumulative provider amount, and reserves pending amounts. The response includes the checked time, scope, refund IDs, and remaining amount.

3. Enable `RESOLVE_PAYPAL_SANDBOX_REFUNDS_ENABLED=true` and `RESOLVE_PAYPAL_SANDBOX_PARTIAL_REFUNDS_ENABLED=true` locally if you intend to move Sandbox funds. Both default to false.

4. Read `GET /cases/{case_id}/reconciliation`. Use its current fingerprint to approve an exact amount up to the recommendation. Then submit a new execution using that approval. After an execution, refresh history and get a new approval for a subsequent amount.

Reconciliation exposes `paypal_refund_history` and `paypal_history_current`. The saved snapshot is current for at most five minutes and can become stale sooner if conflicting local or provider evidence arrives. The execution checks PayPal again before reservation. If history changed, the old approval cannot authorize the new state. Pending refunds, active executions and evidence conflicts block a new execution. Failed/cancelled refunds do not consume balance.

This verifies the records visible for the supported order and app. It does not provide a merchant-wide transaction feed or a provider-side lock. An external refund can race the final POST; PayPal enforces its own remaining balance and may reject the exact amount. See the Orders read (https://developer.paypal.com/api/orders/v2/orders-get) and refund request (https://developer.paypal.com/api/payments/v2/captures-refund) contracts. Unknown outcomes retain a capture hold. A history-gated retry first looks for Resolve's unique `custom_id` in PayPal's records; when the balance changed and no match is found, it blocks another POST. Use the execution's read-only refresh with a known PayPal refund ID to recover an outcome. An unrelated refund of the same amount cannot identify the action.

Receive Sandbox webhooks

Register the HTTPS URL ending in `/webhooks/paypal/sandbox` on the same Sandbox REST app. Subscribe to `PAYMENT.CAPTURE.REFUNDED`, `PAYMENT.REFUND.PENDING`, `PAYMENT.REFUND.FAILED`, and `PAYMENT.CAPTURE.REVERSED`. Put the registration ID in `RESOLVE_PAYPAL_SANDBOX_WEBHOOK_ID` and restart the API.

Resolve sends the original event and signature headers to PayPal's verification endpoint. Only a verified delivery is acknowledged and recorded. Identical event IDs and payloads are deduplicated; a different payload for an existing verified ID is rejected. The inbox stores the digest, event type, matching identifiers and processing state; it does not retain the full event or signature.

From `backend`, run `uv run python -m app.worker --loop` alongside the API. Processing is read-only for webhook work. The worker also recovers existing approved execution records according to their settings and recovery rules. It re-reads PayPal before projecting a refund; reversal notices are held for review. A retry that cannot reconcile a delivery eventually becomes an exception. An authenticated history refresh can clear a matching history exception when the provider records agree.

PayPal's Webhook Simulator (https://developer.paypal.com/api/rest/webhooks/simulator/) does not produce events suitable for postback verification. Use an actual Sandbox event with the REST app's registration to validate the public delivery path. The local Postman simulation exercises the protocol with synthetic events and fake credentials.

Manage a case

Action	Endpoint
Attach evidence	`POST /cases/{case_id}/attachments?filename=receipt.pdf`; send raw bytes with the matching PDF, PNG or JPEG content type
List and download	`GET /cases/{case_id}/attachments`; append `/{attachment_id}` to download
Close or reopen	`POST /cases/{case_id}/close` or `/reopen` with `{"reason":"..."}`
Review exceptions	`GET /review-queue?status=open`; also supports `status=resolved`, `limit`, and `offset`
Resolve a review item	`POST /review-queue/{exception_id}/resolve` with `{"reason":"..."}`

Attachments are immutable, scoped to the case, and capped at 2 MiB each and 20 per case. Resolve checks the file signature and returns the bytes as a download. Closure blocks workflow writes and invalidates consent for that case. An active or uncertain execution must be reconciled before closure; read-only provider evidence can still be updated after closure. Closing a case does not settle a payment or fulfill a commitment.

Review items include failed/unknown executions and PayPal evidence exceptions. A review note can acknowledge a known execution failure or an unmatched event without a capture. It cannot clear an active execution or override conflicting provider evidence. A confirmed reversal requires investigation outside this refund workflow.

Refund execution

Source: docs/REFUND_EXECUTION.md

Sandbox refund execution

Contract and scope

Resolve supports merchant-approved USD Sandbox refunds. Amount and currency are explicit in every provider request. Enable the separate partial-refunds flag to use the verified order history workflow described in PayPal reconciliation (PAYPAL_RECONCILIATION.md).

Execution always requires an open owned case, Sandbox reads and writes enabled, and a live exact-state approval. The first-full path also requires an empty ledger, saved capture evidence and a fresh COMPLETED capture read. The history-gated path requires consistent order/capture/refund records and an exact approved amount within both the outstanding commitment and verified balance. PayPal decides whether the actual request is accepted.

Route	Contract
`POST /cases/{case_id}/refund-executions`	`{request_id: UUID, approval_id: UUID}`; reserve once, submit, and return durable outcome; identical replays return existing execution.
`GET /cases/{case_id}/refund-executions`	List execution records for this case.
`GET /cases/{case_id}/refund-executions/{id}`	Inspect status, request identity, provider result and final capture check.
`POST /cases/{case_id}/refund-executions/{id}/retry`	Recover reserved/unknown or stale submitting work, reusing the exact saved PayPal request ID/body.
`POST /cases/{case_id}/refund-executions/{id}/refresh`	Read a known refund from PayPal and project its latest status; optional `{refund_id: string}` supports read-only reconciliation of an unknown action; no financial POST.
`GET /cases/{case_id}/refund-executions/{id}/events`	Ordered immutable execution history.
`POST /cases/{case_id}/paypal-refund-verifications`	`{refund_id: string}`; independently read a known refund and require its parent capture link to match.
`GET /cases/{case_id}/paypal-refund-verifications`	Saved provider refund observations for the owner's capture.

All routes require normal merchant bearer authentication. Missing/foreign resources are `404`. Invalid inputs are `422`. State/ownership-of-approval, amount, concurrent-lease and stale-state conflicts are `409`. Disabled settings are `503`. A submitted request returns the execution record, including unknown/failed outcomes; callers must inspect status rather than infer success from HTTP status. New execution records return `201`, replays `200`.

Durable state and concurrency

States: reserved -> submitting -> pending/completed/failed/unknown. A request UUID is unique within the merchant. The execution owns one approval and one capture; one active execution per capture prevents competing actions across cases. Reservation snapshots exact consent and final capture evidence in one SQLite transaction. Network calls occur outside the writer transaction.

A 60-second submission lease prevents concurrent local workers from sending the same operation. Unknown outcomes retain a capture-wide hold and use the original PayPal request UUID on recovery, never a newly generated identity. Replays do not submit again. A hash of app client ID, expected seller and sandbox scope binds the idempotency identity;

changing the app/seller blocks financial retries, while rotating a secret for the same app does not. Old records without this identity may only be read-reconciled. A later rejection cannot clear a prior unknown outcome, and stale workers cannot erase a newer submission lease. Recovery POSTs are bounded to 24 hours from reservation, within PayPal's documented refund idempotency period; after that, a known refund can be read but an unidentified outcome requires operator reconciliation.

Known pending/completed/failed provider refunds project into the shared local ledger atomically with execution events and provider observation history. An unknown request is not recorded as a completed or failed refund. Its hold blocks new recommendations/approvals and conflicting manual ledger writes. Provider- managed refund status is refreshed from PayPal rather than manually overwritten. Revocation cannot cancel an already claimed provider action.

Evidence and validation

Use the existing Postman TLS proxy simulation with stateful refund fixtures for success, provider rejection, pending-to-completed, timeout after the provider created a refund, malformed responses, retries, and concurrent calls. Verify persisted request identity, one provider refund, no duplicate ledger projection, restart recovery, stale approval rejection, atomic event rollback and tenant isolation. Existing read simulations remain part of regression validation.

Account-backed validation is limited to the existing USD 1.00 sandbox capture, with an explicitly labelled test commitment and approval. No production payment or real customer agreement is created. A temporary local access key is revoked after checks, and the refund-write setting defaults to disabled outside that test process. The account-backed exercise passed on 2026-11-07: one financial POST, one independent refund read, COMPLETED refund, REFUNDED capture and fulfilled reconciliation. The temporary key and server were cleaned up.

Provider sources: refund endpoint (https://developer.paypal.com/api/payments/v2/captures-refund), refund lookup (https://developer.paypal.com/api/payments/v2/refunds-get), request idempotency (https://developer.paypal.com/api/rest/requests/), and idempotency semantics (https://developer.paypal.com/api/rest/reference/idempotency/).

Aftercare reliability

Source: docs/AFTERCARE_RELIABILITY.md

Aftercare reliability and resolution assurance

CHG-050 implementation contract, 2026-11-10. Extend SDK order support into the nine researched problems using the existing case and payment engines.

Problem	Concrete addition
Return costs / abuse	Versioned merchant return policy, eligibility and quantity checks, disclosed fees, evidence/inspection review flags. Flags require review; they do not declare fraud.
Dispute evidence	Reason-specific evidence requirements and provenance in reviewed packages.
Refund / dispute overlap	Versioned dispute records with stable event IDs and capture-wide holds checked by approval, execution and recovery.
Missing refund	Provider-supported refund journey plus separately labelled merchant trace references and buyer receipt acknowledgement. No claim that processor completion proves bank receipt.
Repeated explanations	One order context with issues, public promises, conversation and next actions, shared with the owned merchant case.
Return instructions / labels	Merchant return guide and authenticated PDF label download/save in the buyer portal and SDK. Label purchase remains external.
Duplicate / out-of-order events	Monotonic order revisions and durable PostgreSQL SDK projection consumer with transaction-bound deduplication.
Missed events	Resumable state-change feed, explicit reconciliation and bounded periodic scan. Consumers can recover current state without relying solely on webhooks.
Ambiguous timeout	Stable-payload retries only on idempotent SDK actions, issue receipt lookup, bounded retry/time budgets; no automatic retry of link rotation or financial actions.

Storage is additive: SQLite 028 / PostgreSQL 019. Keep all earlier numbered migrations unchanged. Source-specific support grants apply to the new customer SDK methods. Staff roles and ownership checks still apply to merchant actions.

The event feed reports reconciled safe state snapshots, not a promise to deliver every intermediate transition. Domain history remains in the existing audit tables. PostgreSQL consumer callbacks should update app rows or an app outbox in the supplied transaction; external side effects still need their own idempotency.

Dispute ingestion may accept merchant-recorded provider references, but labels them as merchant recorded. No new Stripe account integration or live bank-credit observation is implied. Existing authenticated PayPal reads and webhook handling remain provider-specific. Physical return receipt and inspection are distinct from carrier delivery. Real eligibility must also respect applicable merchant obligations; policy configuration is not legal advice or an automatic legal ruling.

Validate synthetic migrations/import, amount precision/shared captures, pending/failed refunds and all payment gates; return policy/revision/label privacy; duplicate and stale dispute events; resumable snapshots; SDK retry ambiguity and PostgreSQL transaction rollback/concurrency; real merchant/buyer mobile paths. Regenerate FastAPI OpenAPI/SDK types and update the engineering log with actual checks and activation limits before handoff. No live money or notifications are part of implementation verification.

Using the workflows

Open **Operations -> Connections -> Return policy and review checks**. An owner can enable a return window, disclose a restocking fee and shipping payer, request item evidence, exclude products from automatic eligibility and require inspection. Unknown or out-of-window dates and repeated requests create review flags, not a fraud verdict. Buyers can still request review. Quantity limits remain hard gates and cannot be overridden by a policy exception. No fee is charged automatically. Policy changes and stale inspections require a current review; an owner/manager can record a documented exception. Inspections remain merchant-recorded findings.

In a case, open **Guided returns and replacements -> Return instructions, saved labels and inspection**. Save the address/instructions and upload an existing PDF return label (maximum 2 MiB). The buyer can download/save it from the private portal and print later. The carrier label must already have been purchased or arranged; Resolve neither purchases postage nor promises a carrier pickup. Item evidence is uploaded through the existing private buyer evidence flow. A clear inspection must refer to the current return and policy revision before its milestone advances.

Open **Dispute safety, refund tracing and evidence requirements** in the case. After linking a payment, managers/finance/owners can record stable, versioned dispute updates. Active or reimbursed disputes hold the shared capture across all its cases. Older events are recorded as ignored; conflicting identities or versions fail. A reimbursement cannot be erased by changing a later status to "no refund." Known PayPal dispute-handoff promises also hold the capture until reviewed. Existing verified PayPal dispute reads remain available through that promise; Stripe/bank references here are merchant-recorded, not live connectors. These holds block new approvals, execution claims and uncertain recovery POSTs. Read-only reconciliation remains possible. A hold cannot recall a payment POST already sent before the hold was recorded.

Finance/owners can add an ARN/STAN/RRN/bank reference obtained from their provider for a refund recorded against this capture. Buyers see a labelled merchant trace reference alongside provider evidence. Provider completion is described as sent to the original payment method, not verified receipt in a bank. Unverified/stale amounts stay null. Buyer acknowledgement is explicitly buyer-reported and changes no payment ledger or approval. Trace data is not automatically retrieved from a bank; availability of a valid reference depends on the payment provider.

Classify original delivery, product description, policy, authorization, cancellation or credit evidence using notes and owned customer attachments. Packages now use the selected dispute reason to list evidence requirements and include labelled documents, return checks and label hashes. Replacement delivery is not silently treated as original delivery. A checklist entry being present establishes only that recorded support is available; it does not verify document authenticity, win a dispute or submit anything to PayPal. Changed sources make older packages stale for review/export.

SDK customer methods

These use the existing explicit support key and source. The merchant app must still check that its authenticated customer owns the requested order.

```
const context = await resolve.getSupportContext(orderId);
const journey = await resolve.getRefundJourney(orderId);
const eligibility = await resolve.getReturnEligibility(orderId, {
  items: [{ product_id: "CUP", quantity: 1 }],
  deliveredAt: trustedDeliveryTimestamp,
});
const returned = await resolve.requestOrderReturn(orderId, {
  requestId: savedReturnRequestId,
  items: [{ product_id: "CUP", quantity: 1 }], reason: "Damaged item",
});
const guide = await resolve.getReturnGuide(orderId, returned.id);
const label = await resolve.downloadReturnLabel(orderId, returned.id, guide.labels[0].id);
```

`getOrderReturns` returns owned item/quantity progress. `lookupIssue(orderId, requestId)` recovers an already-committed receipt after a lost response. `acknowledgeRefundReceived` records the authenticated customer's acknowledgement, not a bank observation. The context includes original requests and a page of the same public case conversation, with an older-message cursor. Imported private support notes and private merchant financial notes stay excluded.

Merchant SDK helpers (`saveReturnPolicy`, `saveReturnGuide`, `uploadReturnLabel`, `recordReturnInspection`, `recordDispute`, `recordRefundTrace`, `recordDisputeEvidence`) require a full merchant key and direct FastAPI URL. Never give these credentials to the buyer or bundle them into browser code.

Reliable synchronization and retries

`pullEvents({ afterSequence, limit })` is a resumable, authenticated state-change feed scoped to the merchant/app source. It reconciles up to 100 least-recently checked owned cases per call; ongoing calls and the separate operations worker cover larger workspaces fairly. Store the returned cursor after processing. The feed includes safe resolution/return state, public conversation version, label/ policy changes and refund journey. It excludes private notes and label bytes.

Run `uv run python -m app.operations_worker --loop` to also reconcile connected app state periodically. This worker sends no customer messages or financial POSTs. The feed can recover missed current state without an inbound webhook; it is not an outbound webhook subscription and does not guarantee every transient state. Domain audit/history retains the original business records.

For a Node app with a writable integration PostgreSQL database:

```
import { PostgresResolutionConsumer } from "@resolve-paypal/sdk";
const first = await resolve.pullEvents();
const consumer = new PostgresResolutionConsumer(pool, {
  merchantId: first.merchant_id, source: first.source, consumerId: "order-page",
});
await consumer.initialize(); // Explicitly creates two SDK tables in this database.
await consumer.poll(resolve, async (event, transaction) => {
  await transaction.query(
    "UPDATE my_orders SET support_state=$1 WHERE id=$2",
    [event.snapshot.resolution.stage, event.order_reference],
  );
});
```

Use a stable consumer ID. The consumer serializes concurrent passes, ignores processed events and older order revisions, rejects scope/revision conflicts, and commits app writes, its projection and cursor together. A callback failure rolls them all back. Callbacks must use the supplied transaction; enqueue external notifications in the app's own outbox there. Direct external effects still need independent idempotency and cannot be made exactly-once by this SQL transaction. The SDK has no `pg` runtime dependency; the host app supplies its existing pool.

SDK retry-safe actions get at most three attempts by default (`retryAttempts` 1-4), preserving the serialized payload and request ID within the overall timeout. Network errors and short 429/5xx responses are retried; authentication, validation, conflict, long rate limits and cancellation return to the caller. Portal-link rotation and merchant mutations are not automatically retried. On an exhausted request deadline, use the same durable request ID or `lookupIssue`; never invent another ID to resolve an unknown outcome. No SDK retry executes a refund.

Verification and activation

All development uses disposable databases and synthetic accounts. The engineering log records actual checks, provider simulation boundaries, schema versions and browser results. New migrations are SQLite 028/PostgreSQL 019, and SDK 0.7 remains a local preview archive. No live bank account, new payment connector, postage purchase, customer notification, hosted deployment or dispute submission is implicitly configured by installing the SDK.

Active return policies disclose the delivery window, shipping payer, restocking fee and evidence/inspection requirements before a buyer submits a request. The capability-protected public view omits the merchant's excluded-product catalog. An inactive policy returns a null fee and asks the merchant to confirm costs; it does not advertise free returns. Eligibility remains advisory until approval.

Embedded aftercare

Source: docs/SDK_AFTERCARE.md

Embedded order aftercare

Implemented for CHG-049, 2026-11-10: give the SDK a complete customer-support workflow inside a merchant's existing order page. The server authenticates its customer and verifies order ownership before calling Resolve. A dedicated support key explicitly grants this workflow for one app source; existing commerce-only keys retain their restrictions. No app-supplied complaint becomes an approved merchant promise or authorizes a refund.

Workflow contract

1. Existing `syncOrder` publishes the order from trusted server data.

2. `openIssue({orderId,requestId,reason,message,requestedResolution})` atomically

opens/reuses the order's case and records one customer issue/message. Retries reuse the same request ID; changed requests conflict. No duplicate case/message.

3. `getOrderResolution(orderId)` returns a customer-safe progress summary, current

promised remedies, overdue flag, refund verification/freshness and next action. It returns `no_issue` for a known order with no case, without creating one. Unknown orders are 404; private merchant evidence/notes are never returned.

4. `createSupportLink(orderId)` explicitly issues an expiring buyer capability

link for that existing case. It replaces the prior case link, like the current merchant issuer. Buyers can then use offers, messages, uploads and returns in the existing portal. Session creation is an explicit action, not status polling.

API: POST `/aftercare/issues`, `/aftercare/status`, `/aftercare/portal`, JSON order locator `{source,order_reference}`. All are bounded, private/no-store and available through the server-authenticated `/api/sdk` gateway. Status is a read-only POST to keep private order references out of URLs. No provider refresh occurs while polling. The merchant can refresh existing provider evidence separately.

Storage: new immutable SQLite 027 / PostgreSQL 018 migrations add explicit per-key support grants and idempotent issue receipts. Existing schemas and old commerce permissions remain intact. Owner-created support keys are still unable to access general merchant operations, financial execution or team management.

The SDK/server tests and simulated store browser flow cover the actual use case, with disposable data. The store simulation passed with 19 mocked provider requests and zero pass-through. CHG-049 in the engineering log records final commands, counts, migration/import validation and remaining limits.

Merchant setup

Open **Operations -> Connections -> Customer support inside your store**. Choose the same source your app uses when syncing orders, generate a support integration key, and copy the server settings. Old order-only keys deliberately remain order-only. The support grant is source-specific; the underlying key retains the existing commerce permissions. Revoke it using the same key list. Already issued buyer links have their own expiry/revocation; revoking a server key stops further API requests but does not revoke an existing buyer capability.

Install the local SDK 0.6 preview archive or use the repository's linked package. The package is not published to npm. Keep integration code in the app's backend, never bundle the key into a storefront. This feature needs no new environment secrets or worker. Existing workers and optional provider connections still perform their existing tasks. `getOrderResolution` reads cached evidence and makes no external request; it is safe to use when opening an authenticated order page.

What the customer sees

Status field	How to use it
`stage: no_issue`	A known order has no case; offer "Report a problem." Reading status creates nothing.
`stage: under_review`	The case is with the merchant. Show `next_step.action` and any due date.
`next_action: review_offer`	A current offer awaits the buyer. Show "Review offer," then create/open a portal link.
`next_action: open_portal`	The merchant needs a customer update. Show the messages/returns entry point.
`stage: pending`	The recorded resolution is awaiting its next result. Display the supplied next step.
`stage: needs_review`	Evidence needs review; avoid promising an outcome.
`stage: resolved`	Current promised remedies are fulfilled according to their evidence sources.
`promises[].provider_confirmed`	Distinguishes provider-confirmed completion from merchant-recorded completion.
`refund.verification: current`	Complete provider history is fresh (at most five minutes) and the owning connection has not changed. Completed and pending amounts are separate.
`refund.verification: unverified`	Completed/pending/remaining amounts are null. Show that a check is needed, not "no refund."

All results have `authorizes_payment: false`. Customer requests are not promises; merchant review and existing approval/condition/acceptance gates remain separate. No app endpoint executes a refund or replacement. Status contains public promise terms, not private notes, payment identifiers, raw sources or conversations. Portal sessions deliberately expose the customer's existing case conversation, attachments, offers and return controls, just like ordinary private buyer links.

Runnable use case

The Field Brew store (../examples/checkout-store/README.md) now shows a **Need help with this order?** section on a paid, synced receipt. Its own HttpOnly browser session scopes every support action to the owned order. The server calls `openIssue`, `getOrderResolution` and `createSupportLink`; the browser never sees the SDK key. It persists a request UUID for identical retries. The production app integrating Resolve should use its own account/order authorization and durable retry storage in the same way.

The sample covers damaged/not-received/wrong-item/refund-not-received reports, requested refunds/replacements/returns/advice, status refresh, and portal entry. It coordinates the current merchant workflow; actual physical fulfillment remains merchant-arranged and provider verification is presently PayPal Sandbox only.

Merchant case details also show **Customer requests from connected apps** with the reported problem, requested remedy, app source and date. This is separate from the merchant's promises and approvals. The private merchant projection is `GET /platform/cases/{case_id}/support-requests`; support keys cannot read it.

Automatic investigations

Source: docs/AI_INVESTIGATIONS.md

Automatic investigations

Verified-snapshot reads and real filming evidence (12 November 2026)

CHG-060 fixes an observed false stop with actual Groq triage. If the model selects a linked capture/refund that is already independently verified, the server can satisfy the read from the same case's matching resource within the smaller of five minutes and `RESOLVE_AI_FRESHNESS_SECONDS`. Ownership/type/reference checks run first. A recent failed refresh, stale observation, future timestamp, foreign link or missing complete refund history cannot be promoted to verified evidence. The existing revision/lease/snapshot checks still run before publication; a successful reuse adds `investigation_read_cache_hit` to the private runtime audit. It never creates a refund approval or execution.

The local filming run now verified the actual existing worker with official Sandbox evidence and Groq `openai/gpt-oss-120b`: three provider findings, state `finished`, confirmed full refund, next step `no_action`, zero validation failures and zero financial writes. Earlier 20B/120B runs that stopped on redundant fresh reads are preserved as failed-run evidence. Targeted regression: 41 passed; full isolated backend: 1,008 passed, 63 opt-in skipped. These are local/Sandbox observations, not hosted acceptance or validation of every model/route.

The owner's local key exists; process-only settings and an intentionally secret-isolated filming service previously did not inject it. AI settings require explicit process-environment injection; merely editing the ordinary dotenv file does not override this boundary. The isolated capture opts into minimized standard retention for structured test-case facts, with contacts/messages/payment IDs excluded. Owner production policy is unchanged. Actual OpenRouter escalation, formal external dispute submission and autonomous payment authorization are not claimed. The investigator proposes typed reads/recommendations, not arbitrary network or financial tools.

Audit and implementation plan (2026-11-08)

Usable: FastAPI `/resolution-cases`, `/resolution-cases/upsert`, evidence `records`, `events`, `refresh`, evidence-bundle and cursor timeline; merchant bearer/browser identity; transactional SQLite/PostgreSQL stores; separate refund approval/execution; independent recovery/follow-up workers. SDK methods actually present: createCase, upsertCase, linkRecord, addEvidence, refreshEvidence, getCase, getTimeline, getEvidenceBundle, syncOrder. Commerce and sign-in changes already in the working tree are preserved. Provider observations preserve prior facts when a refresh fails. Partial: connectors need account-backed verification and workers need an operator. Missing before this change: model adapters, automatic investigation queue, structured recommendations and human investigation review.

1. Entry point: transactional enqueue on resolution creation and relevant

evidence writes; coalesce repeated requests by revision. Use a separate bounded worker, with leases and revision checks, never inference in requests.

2. Evidence: owned resolution events, linked records, structured merchant

approval claims, independently verified PayPal Sandbox facts and freshness, linked legacy payment reconciliation. Free text and contacts stay local.

3. Groq selects issue/read/escalation; OpenRouter investigates ambiguous cases.

Server validates citations and builds factual findings; model output grants no authority. Unavailable inference produces a blocked reviewable result.

4. Approval boundary: full merchant identity reviews a revision and snapshot.

Investigation approval is advisory and never creates a refund approval.

5. Acceptance: labeled synthetic refund states and adversarial cases; automatic

queueing, duplicate/coalesced triggers, leased workers, bounded retries and reads, stale/concurrent/repeated review, failed refresh preservation, tenant isolation, forged citations, injection and no financial side effects.

Implementation details, model policy, commands and validation follow below.

Model decision and official evidence

The current OpenRouter selection was checked on 2026-11-10 using public, unauthenticated catalogs and the ZDR endpoint listing (CHG-047). The owner requires output pricing strictly below USD 0.75 per million tokens. This supersedes CHG-046's Gemini default, which exceeded that output-price limit.

Candidate and route	Structured output	Published input/output USD per million tokens	Decision
Groq `openai/gpt-oss-20b`, direct Groq	Strict JSON schema	0.075 / 0.30 (previous catalog check)	Unchanged default triage
Groq `openai/gpt-oss-120b`, direct Groq	Strict JSON schema	0.15 / 0.60 (previous catalog check)	Unchanged optional triage model
OpenRouter `openai/gpt-oss-120b`, `phala`	Endpoint lists `structured_outputs`, `response_format`, `max_tokens`, `reasoning_effort`	0.15 / 0.60	Default deeper investigation; low reasoning effort
OpenRouter `xingchen-agi/xing4.0-29b-a4b:free`, `phala`	Endpoint lists `structured_outputs`, `response_format`, `max_tokens`	0 / 0	Optional free model, manual selection

The GPT-OSS 120B route catalog (https://openrouter.ai/api/v1/models/openai/gpt-oss-120b/endpoints) confirmed Phala's exact prices and schema parameters; the ZDR endpoint listing (https://openrouter.ai/api/v1/endpoints/zdr) included that route. GPT-OSS 120B is the larger reasoning-capable paid choice within the specified budget; this is not measured evidence of better quality on Resolve. The free Xing4.0 route catalog (https://openrouter.ai/api/v1/models/xingchen-agi/xing4.0-29b-a4b:free/endpoints) provides the optional zero-token-price alternative on the same pinned provider.

Runtime defaults and the private `.env.ai.ps1` select `openai/gpt-oss-120b` with `RESOLVE_AI_OPENROUTER_ROUTES` set to `["phala"]`. For free use, manually set `RESOLVE_AI_OPENROUTER_MODEL` to `xingchen-agi/xing4.0-29b-a4b:free`. The default allowlist contains only these two choices. Gemini and Sonnet are excluded. No automatic model fallback is permitted.

Each OpenRouter request includes `provider.max_price={"completion":0.74}`. OpenRouter documents these values in USD per million tokens in its maximum-price routing controls (https://openrouter.ai/docs/guides/routing/provider-selection#max-price). This keeps the requested output token tariff strictly below 0.75 even if a provider raises its listed price. When no route satisfies pricing, privacy or schema requirements, the existing adapter blocks the investigation. This is a provider token-price filter, not a total-account-spend or platform-fee limit. Account-backed billing and real model quality have not been verified.

Groq triage is unchanged; its catalog (https://console.groq.com/docs/models), strict schema documentation (https://console.groq.com/docs/structured-outputs) and rate limits (https://console.groq.com/docs/rate-limits) describe its controls. Use the fixed synthetic evaluator for each new OpenRouter model before enabling real inference. Offline simulation validates the pipeline and routing, not the quality, latency or real availability of these models.

OpenRouter structured output documentation (https://openrouter.ai/docs/guides/features/structured-outputs) and routing controls (https://openrouter.ai/docs/guides/routing/provider-selection) support `only`, `allow_fallbacks=false`, `require_parameters=true`, `data_collection=deny`, and `zdr=true`. Every deeper request supplies all of them. A response from

another provider, missing provider identity, an unsupported parameter, or no eligible route blocks the investigation. Catalog parameter support does not prove current ZDR eligibility or account availability; runtime routing must enforce it. No fallback to broader routes is permitted.

Groq data handling (https://console.groq.com/docs/your-data) distinguishes usage metadata from inference content. Limited reliability/abuse retention may occur unless organization data controls opt out; there is no request-level ZDR switch. Therefore default `data_policy=zdr_required` blocks direct Groq and the split fails closed. `minimized_standard_retention` explicitly accepts that documented Groq policy while still requiring ZDR at OpenRouter. Unknown policies block. For strict ZDR requirements, obtain a verifiable Groq organization policy and implement its enforcement before enabling this split. A local boolean would not prove that control, so one was deliberately not added.

Runtime, evidence and approval boundaries

SQLite migrations 021-025 and PostgreSQL 010-015 append durable jobs, results, reviews and safe audit records. Existing cases are backfilled. Evidence events, links, customer messages/files, commerce updates and linked financial/provider changes increment a case evidence revision transactionally. Duplicate API evidence replays do not write again. Many writes coalesce to the latest revision. Shared-capture changes invalidate every owned linked resolution case.

The first authored investigation migration referred to the snapshot table removed by commerce migration 020/009. Its SQL was retained unchanged. The runner applies `investigation_prerequisite.sql` immediately before that migration to create an empty transitional table; the immediate forward correction 022/011 removes it and installs the current commerce trigger, in the same transaction. No existing numbered migration/checksum was rewritten. PostgreSQL 014 defers the new import foreign keys, and the verified importer disables only investigation triggers during restore, flushes constraints, re-enables triggers, and verifies copied values. This preserves stored revisions rather than re-enqueueing restored history.

Worker claims use the existing serialized write transactions, a three-minute lease token, revision checks and a maximum of two inference attempts per revision. A third claim after repeated worker crashes publishes a blocked result without inference. Late workers cannot overwrite a new revision or lease. There are at most four HTTP attempts per worker attempt (eight over crash recovery), at most one selected evidence refresh, one retry per provider call, 1800 default output tokens (maximum 2400), 32 KiB request and 64 KiB response limits. HTTP timeouts default to 12 seconds (maximum 20); adapter deadlines also bound a trickling response. Worker passes default to 20 jobs, maximum 100. There are no model tools, arbitrary URLs, financial operations or general agents.

The allowlist is `refresh_linked_capture` / `refresh_linked_refund` with an existing evidence-link UUID. A server callback checks ownership, link kind, merchant/seller configuration, capture amounts and refund parent. One resource GET plus OAuth is the maximum; a refund needs an already verified linked capture. Automatic reads are separately disabled by default. A failed read retains the actual earlier verified time. A read updates the evidence revision; the old run is superseded and the next run sees the new snapshot. Attempts less than five minutes old cannot be repeated. A single GET cannot re-establish complete order history, so its result conservatively needs review/full history.

Findings are built by application code from existing source event/verification IDs, decimal amounts, supported statuses, capture relationships and UTC times. Duplicate identities, excessive refunds, future timestamps, inconsistent capture statuses and missing history block confident conclusions. Failed refunds do not consume completed balance; pending refunds remain pending. The manual ledger and merchant approval claims never become provider facts. Existing acceptance, condition, prerequisite, execution and provider-review gates remain visible as missing evidence or contradictions. Existing independently verified payment history is usable without a duplicate SDK refresh; its five-minute history expiry also caps investigation freshness and invalidates advisory approval. Model recommendations must cite existing findings and agree with validated status/next-step enums or explicitly abstain. Free-form model factual prose and monetary execution proposals are intentionally not part of the accepted schema.

Notes, titles, source messages, support ticket text, conversations, attachments, contacts, provider bodies and payment identifiers are not sent to inference. Structured claims, facts, timestamps, source UUIDs and authorized link kinds are sent. This reduces extraction breadth: free-text commitment extraction and non-refund remedy investigation remain future work. Input-budget overflow, unavailable inference, malformed/refused/truncated outputs and invalid citations produce a

private, reviewable blocked result. Raw prompts/responses never enter operational logs; audit records contain IDs, revision/hash, model/route, bounded usage metadata, structured assessments and validation codes.

The merchant panel appears below the selected case, with queued/running state, the latest completed or blocked result, findings and evidence IDs, freshness, contradictions, missing evidence, abstention/proposal, and review actions. It polls every 15 seconds. Reviews record merchant plus a pseudonymous full-key or session identity, timestamp, decision, revision and snapshot. Identical decisions from that identity are idempotent; conflicting concurrent reviews have one winner. Superseded snapshots cannot be reviewed. Expired evidence cannot be approved; a reviewer can still reject/request evidence on the current snapshot.

`POST /resolution-cases/{case_id}/investigations/{id}/review` accepts `approve`, `reject`, or `request_evidence`, with `expected_revision` and `evidence_snapshot`. It always returns `authorizes_payment=false`. It never creates a refund approval, execution, customer message or provider write. Only the pre-existing explicit refund approval/execution routes can move money. Full merchant authentication is required for list/review; commerce-scoped keys and buyer links cannot access these results. Buyer payloads do not include investigation fields.

Configuration and commands

Run these commands from `backend`. `scripts/isolated.py` clears inherited Resolve, provider and database configuration by variable name before imports, prevents the global FastAPI app from reading dotenv, and creates a fresh workspace-local test database. It also avoids Python's special Windows 0700 ACL in this managed sandbox. It never examines existing secret values.

```
$env:UV_CACHE_DIR = "$PWD/../.local/uv-cache"
uv run --no-sync python scripts/isolated.py -m pytest -q -p no:cacheprovider
uv run --no-sync python scripts/isolated.py scripts/evaluate_investigations.py
uv run python scripts/isolated.py scripts/export_openapi.py
```

Direct `.venv/Scripts/python.exe` is an equivalent fallback if uv cannot access its default cache. The exporter command requested by AGENTS is still `uv run python scripts/export_openapi.py`; use the isolated wrapper in this run because importing the pre-existing global app otherwise reads backend dotenv.

PostgreSQL validation used a newly initialized local cluster under `.local/pg-ai/data`, bound only to `127.0.0.1:56439` with the synthetic `resolve_synthetic` role. With that disposable cluster running, use `uv run python scripts/isolated.py --postgres-local --storage-postgres -m pytest tests/test_investigations.py tests/test_postgres.py -k "not contended_writer" -q`. Each test creates and drops a separate schema. The excluded legacy lock-timeout test hung in this managed host and needs rerunning with normal process signaling; the feature concurrency and migration/import tests ran against actual local PostgreSQL. Never point this validation at the owner's storage. The managed host denied shutdown after this run. Stop its remaining synthetic cluster from a normal terminal with `& 'C:/Program Files/PostgreSQL/18/bin/pg_ctl.exe' -D 'C:/Users/navne/Desktop/paypal-resolve/.local/pg-ai/data' -m fast -w stop`.

Normal worker operation (operator-selected storage, environment injection only):

```
uv run python -m app.investigation_worker            # one bounded pass
uv run python -m app.investigation_worker --loop     # every 10 seconds
```

Inference stays off unless `RESOLVE_AI_ENABLED=true`. All AI settings use the `RESOLVE_AI_` prefix and never read dotenv. Configure `GROQ_KEY`, `OPENROUTER_KEY`, `GROQ_MODEL`, `OPENROUTER_MODEL`, `APPROVED_GROQ_MODELS`, `APPROVED_OPENROUTER_MODELS`, `OPENROUTER_ROUTES`, `DATA_POLICY`, `ALLOW_SANDBOX_REFRESH`, `TIMEOUT_SECONDS`, `RETRIES`, `MAX_TOKENS` and `FRESHNESS_SECONDS` under that prefix. Model/route tuple values use JSON arrays. Keep credentials in the server process secret store. No browser model keys. Storage and optional Sandbox credentials retain the existing `RESOLVE_` settings; this worker does not load the owner's backend `.env`. It is separate from both the refund recovery worker and hosted follow-up worker.

The TypeScript SDK adds `getInvestigations(caseId)` and `reviewInvestigation(caseId,id,input)`. They require the direct FastAPI base URL and a full merchant credential; the commerce SDK gateway intentionally excludes them. `npm run build` regenerates SDK types from the OpenAPI contract.

Evaluation and release evidence

`backend/evaluation/investigations.json` sets the rubric **before comparison**: factual accuracy >=95%, citation validity 100%, correct abstention 100%, and recommendation quality >=90%. Correct status, real owned citations, expected abstention and appropriate next-step enums are scored separately. Deterministic gates require zero tenant disclosure, unauthorized action, accepted fabricated citation or approval bypass. These gates also require the auth/concurrency pytest suite; model scores cannot waive a failed gate.

The 13 labels cover full, partial, pending and failed refunds, approval without confirmation, contradictions, stale/missing evidence, provider failure, forged citations, cross-tenant access, injection and simultaneous partial/pending amounts. The offline evaluator runs the real adapters and worker against loopback HTTP stand-ins. Its 100% pipeline scores show conformance to those stand-ins, not model accuracy or real provider latency. Final factual fields are server-derived. Raw structured model assessments are recorded separately for future comparison; independent model grounding quality is not established by pipeline accuracy.

Optional **live synthetic only**, never run during this build:

```
# Inject new evaluation-only keys under these exact names in a separate terminal:
# RESOLVE_SYNTHETIC_EVAL_GROQ_KEY, RESOLVE_SYNTHETIC_EVAL_OPENROUTER_KEY
uv run python scripts/evaluate_investigations.py --live --confirm-synthetic-only
uv run python scripts/evaluate_investigations.py --live --confirm-synthetic-only --groq-model openai/gpt-oss-120b
--openrouter-model openai/gpt-4.1-mini
```

This command constructs only the fixed labeled cases in a new SQLite file. It accepts no database, customer, case, prompt or evidence input; it never imports the configured FastAPI app or constructs PayPal. Separate eval keys are the only keys read. Network fault/forged-output labels remain explicitly simulated even in live reports. Reports under `.local/ai-evaluation/` record model/route, structured assessments, latency, token usage and estimated known token cost. Failed-call usage and platform fees may be unknown. No model benchmarking was performed here. Run on these labels, inspect false abstentions/invalid outputs, and pass deterministic gates before enabling any merchant inference.

Browser test: `npx playwright test -c playwright.investigations.config.ts` from `frontend`. It starts isolated FastAPI and a Next source copy excluding dotenv and build caches. `npm run check` checks the surface without starting it. The native HTTP gateway test is `node --test --experimental-test-isolation=none tests/investigations.gateway.mjs` after those isolated servers are running. On hosts that block Node subprocesses, start FastAPI with `../backend/.venv/Scripts/python.exe ../backend/scripts/isolated.py tests/fixture_backend.py` from `frontend`, then `node tests/isolated-next.mjs --in-process` in a second terminal. That fallback uses compiler worker threads in the disposable copy. For another loopback port, pass `--port=3109` and set `RESOLVE_AI_UI_TEST_ORIGIN=http://127.0.0.1:3109` for the gateway test. The SDK fixture can be started from `backend` with `.venv/Scripts/python.exe scripts/isolated.py ../sdk/typescript/test/fixture.py ../.local/sdk-ai.sqlite3 --investigations --descriptor ../.local/sdk-ai-fixture.json`; then run `node --test --experimental-test-isolation=none test/investigations.external.mjs` from `sdk/typescript`. Start a fresh fixture for each SDK integration run. Browser/visual verification remains required when the host permits launching Playwright. Postman CLI was not available on PATH in this run; the local Python HTTP stand-ins avoid account setup, proxy passthrough and live provider calls. Existing Postman financial mocks/contexts were preserved. No deployment, hosted storage, real inference, real Sandbox request or financial write was used.

Server-side SDK

Source: sdk/typescript/README.md

Resolve Evidence SDK for Node.js

SDK **0.7** adds return policy checks, saved label downloads, inspection review, reason-specific dispute evidence, capture-wide dispute holds, refund tracing, shared support context, issue receipt recovery and resumable app state updates. See the assurance guide (../../docs/AFTERCARE_RELIABILITY.md) for setup and limits.

Put order support inside your store. SDK 0.6 turns your order page's "Need help?" action into a tracked case, a customer-safe resolution status and a private portal for messages, offers and returns. Your customer stays connected to the same order while the merchant works in Resolve.

An example: a buyer reports a broken cup and requests a replacement. Your server opens the issue once; the merchant sees it in the inbox, approves a remedy and publishes an offer. Your app shows the next action, and the buyer opens the portal to accept the offer or request a return. Repeated network requests do not create duplicate issues. A requested refund is separate from a merchant promise and payment approval.

Create a **support integration key** under **Operations -> Connections** for your stable app source. Save its copied settings on your server. Authenticate the customer and verify that they own the order before every support call:

```
import { createResolve } from "@resolve-paypal/sdk";

const resolve = createResolve();

// Run after the app saves the paid order, using its trusted order data.
await resolve.syncOrder({ id: "ORDER-123", status: "paid" });

// Run after your server checks that the customer owns ORDER-123.
await resolve.openIssue({
  orderId: "ORDER-123",
  requestId: savedComplaintId, // A UUID persisted by your app; retain on retries.
  reason: "damaged_item",
  message: "The cup handle arrived broken.",
  requestedResolution: "replacement",
});

const progress = await resolve.getOrderResolution("ORDER-123");
// Render progress.stage and progress.next_step.action on your order page.

// On an explicit "Open support" action; return this only to the owning customer.
const portal = await resolve.createSupportLink("ORDER-123");
```

The default environment keys are `RESOLVE_API_URL`, `RESOLVE_API_KEY`, and `RESOLVE_APP_SOURCE`. Use the same source as the support key and order sync. Support calls work through `/api/sdk` or direct FastAPI. Existing commerce-only keys need an explicitly issued support key; old permissions do not expand. The support grant is limited to its app source; regular commerce permissions remain unchanged. Keep the key on the server. Full workflow, status meanings, limits and runnable store example (../../docs/SDK_AFTERCARE.md).

Merchant operations helpers include `getOperationsDashboard`, `getMissedPromises`, `scanMissedPromises`, `importZendeskTicket`, `confirmSupportImport`, `publishResolutionOffer`, `getReturnWorkflow`, `requestReturn`, `updateReturn`, `prepareDisputePackage` and `reviewDisputePackage`. Inputs and results use the generated OpenAPI types. These helpers require a **full merchant key and direct FastAPI URL**; commerce keys and `/api/sdk` do not expose them. Provider checks remain explicit and offer acceptance never authorizes payment. See the workflow guide (../../docs/PLATFORM_EXPANSION.md).

Private investigations: `getInvestigations(caseId)` returns queued/running state and completed or blocked results. `reviewInvestigation(caseId,id,{expected_revision, evidence_snapshot,decision})` records `approve`, `reject` or `request_evidence`. The receipt always has `authorizes_payment: false`; stale snapshots return 409. These methods require a

full merchant key and direct FastAPI URL. Scoped commerce keys and the `/api/sdk` gateway do not expose investigation review. See automatic investigations (../../docs/AI_INVESTIGATIONS.md).

`@resolve-paypal/sdk` calls the Resolve API from your server. It lets a support agent see what happened, what evidence supports it, and what is still missing. It can also issue a private status link for the buyer. It needs a running Resolve API and the appropriate merchant key for the workflow. Node.js 22.12+ and TypeScript types are supported. The package has no runtime dependencies.

Integrate with your IDE assistant

Copy the resolve-sdk-integration skill (skills/resolve-sdk-integration/SKILL.md) into your app's skill folder, keeping it at `resolve-sdk-integration/SKILL.md`, or attach the file to your IDE chat. Give the IDE the path to the SDK preview archive if the SDK is not installed in your app yet. Then paste:

```
Use the resolve-sdk-integration skill to connect this app to my Resolve workspace.
Find where the server saves orders or payment updates and add the smallest
server-side sync so orders appear in Workspace  ->  Orders. Reuse my existing
checkout and keep the SDK key in the server environment. Show me which settings
to add and how to verify one test order.
```

Copy the real settings from **Workspace -> Settings -> SDK keys** to your server environment outside the IDE chat.

Install this preview

The package is not published to npm. From the Resolve checkout:

```
cd sdk/typescript
npm ci
npm pack
```

In your own application's backend, install the resulting `resolve-paypal-sdk-0.7.0.tgz`:

```
npm install path/to/resolve-paypal-sdk-0.7.0.tgz
```

Quick start from the merchant dashboard

Sign in with your Resolve access key, then open **Settings -> SDK keys**. Select **Generate SDK key**, then **Copy SDK setup**. Paste the configuration into your app server's environment. It includes the SDK gateway URL and the new key; keep it out of browser code and Git.

From your server (for a standalone Node script, load the file with `node --env-file=.env your-script.mjs`):

```
import { createResolve } from "@resolve-paypal/sdk";

await createResolve().syncOrder({ id: "ORDER-123", status: "paid" });
```

That registers revision 1 in **Workspace -> Orders**. For updates, pass the order's next version, for example `version: 2`. Keep the same order ID and app source. `payment` and `items` are optional; include them if you need PayPal comparison or line items. For more than one app, give each a stable `RESOLVE_APP_SOURCE`. The copied key only works for commerce actions. To replace it, generate a new key, update your app, then revoke the old one.

If you already have a PostgreSQL reader pool, use `createResolve().connectPostgres(pool).register(orderId)` or `.registerAll()` for an initial import. The full integration guide (../../docs/COMMERCE_INTEGRATION.md) shows the database views and how to resume a backfill.

Connect an app that already accepts PayPal

For the full setup, use the commerce integration guide (../../docs/COMMERCE_INTEGRATION.md). With the read-only PostgreSQL views (examples/postgres-views.sql) and a dedicated commerce API key, the SDK can sync one order and check PayPal in one call:

```
const evidence = resolve.connectPostgres(readOnlyPool, { source: "my-store" });
```

```
const report = await evidence.inspect("STORE-2042");
```

Use `syncCommerceOrder()` directly for other databases or ORMs, `getCommerceOrder()` to read the latest comparison without calling PayPal, and `refreshCommerceOrder()` to check again. Contacts are opt-in and full merchant access is required to read them via `getCommerceContact()`. The commerce key cannot call the general case or buyer-link methods below; those retain their existing full merchant credentials.

Run the PostgreSQL example (examples/connect-postgres.mjs) after installing `pg` in your app. The SDK itself has no runtime dependencies.

To show routine orders in the dashboard, use `registerCommerceOrder(snapshot)`, `evidence.register(orderReference)`, or the paginated `evidence.registerAll()` backfill. These methods do not create a case for an order that doesn't have one. The full order dashboard workflow (../../docs/COMMERCE_INTEGRATION.md#show-all-orders-in-the-merchant-dashboard) includes a resumable example and the endpoint contract.

Add Resolve to your backend

Keep these values in your application's server environment:

```
RESOLVE_API_URL=http://127.0.0.1:8000
RESOLVE_API_KEY=your_resolve_merchant_access_key
```

`RESOLVE_API_URL` is the FastAPI base URL. Use HTTPS for a hosted API. Issue a merchant key from the Resolve checkout with `uv --directory backend run python -m app.manage issue-key --merchant-id YOUR_MERCHANT_UUID`. Keep the key out of browser code and public environment variables. Verify the connection with `await resolve.getMerchant()` after constructing the client.

For provider refreshes, use a Resolve backend with the PayPal Sandbox read settings (../../backend/README.md#paypal-sandbox-capture-reads) configured for that merchant. The SDK itself does not receive PayPal credentials or require the refund write flags.

```
import { ResolveClient } from "@resolve-paypal/sdk";

const resolve = new ResolveClient({
  baseUrl: process.env.RESOLVE_API_URL!,
  apiKey: process.env.RESOLVE_API_KEY!,
});

export async function inspectRefund(orderReference: string, captureId: string) {
  // Check that the caller can act on this order in your app first.
  const supportCase = await resolve.upsertCase({
    external_reference: orderReference,
    order_reference: orderReference,
    title: "Refund inquiry",
  });
  await resolve.linkRecord(supportCase.id, { kind: "paypal_capture", reference: captureId });
  await resolve.refreshEvidence(supportCase.id);
  return resolve.getEvidenceBundle(supportCase.id);
}
```

Run examples/order-1042.mjs after setting `RESOLVE_API_URL`, `RESOLVE_API_KEY`, and `PAYPAL_CAPTURE_ID` in your server environment. `PAYPAL_ORDER_ID` is optional. Run `node node_modules/@resolve-paypal/sdk/examples/order-1042.mjs` from the consumer app, or `node examples/order-1042.mjs` after building in this checkout. The example records a USD 25 approval as merchant evidence and shows which PayPal facts are available. It does not issue a refund.

Method	What it does
`upsertCase(input)`	Uses a stable external reference per merchant. Save the returned `id`. Reusing the reference with different case data returns 409.
`linkRecord(id, input)`	Associates a PayPal order, capture, or refund ID, an internal order, or a support ticket. A link alone proves no provider outcome.

Method	What it does
`addEvidence(id, input)`	Records an approval, return confirmation, or private note with a source, event ID, and event time. It is merchant-reported evidence.
`refreshEvidence(id)`	Reads PayPal Sandbox and returns newly checked facts, check time, and evidence issues. A failed check preserves earlier facts.
`getCase(id)` / `getEvidenceBundle(id)`	Reads the case, evidence summary, and last check. The bundle adds links and a page of evidence timeline events.
`getTimeline(id, options)`	Orders case events newest first by sequence. Pass `next_before_sequence` as `before_sequence` for the next page of `items`; legacy `refund_items` is a separate stream. Existing offset pagination still works.
`createBuyerLink(id)`	Uses the existing private link issuer. It replaces the previous link.

Identical upserts, links, and evidence events return their existing records. Keep event IDs stable per case and source. Changed input under the same key returns 409; `ResolveApiError.code` distinguishes the conflict types. Use decimal strings such as `"25.00"` for USD amounts.

PayPal Sandbox verification is scoped to the configured merchant and supports one USD capture per case. If the capture response identifies its PayPal order, Resolve tries to discover refunds from that order; otherwise link the PayPal order ID explicitly. A known refund can be checked against its verified parent capture even when order history is unavailable. Refund-only links without a verified capture and non-PayPal references are reported as missing or unsupported. The SDK distinguishes pending amounts, completed amounts, and partial refunds without treating merchant approval as provider proof. A provider check cannot move money or change a merchant approval.

Set `RESOLVE_CUSTOMER_PUBLIC_ORIGIN` or `RESOLVE_MERCHANT_WEB_ORIGIN` on Resolve's backend so it can return `public_url`. Otherwise it returns the one-time token with a null URL. Send the link only to the intended buyer; anyone with the link can open its customer view until it expires or is revoked. Issuing a new link replaces the old one, and `revokeCustomerLink(id)` disables it. `getCustomerLink(id)` returns its status without disclosing the token. The buyer sees safe status labels only; private notes, provider IDs and merchant evidence remain in the merchant workspace.

`getMerchant()` checks access. `listCases({ limit, offset })` and `getCaseTimeline(id, { limit, offset })` read merchant-only records. `ResolveApiError.status` exposes HTTP failures; `ResolveTransportError.code` distinguishes network, timeout, cancellation, and invalid responses. Requests have a default 10-second timeout and can take an optional `AbortSignal`. The SDK doesn't retry requests. If a create or link request times out, check whether it succeeded before repeating it.

Other languages can call the same FastAPI REST endpoints using the repository's OpenAPI contract (../../postman/contracts/resolve.openapi.json).

Develop and verify

Sync the Resolve backend with `uv --directory backend sync` from the repo root. Run `npm test` here to generate TypeScript types from the checked-in OpenAPI contract and exercise the SDK against real FastAPI with disposable SQLite and synthetic keys. No configured database or PayPal account is used. From the repo root, run the ORDER-1042 acceptance test through Postman:

```
$env:RESOLVE_RUN_SDK_EVIDENCE_SIMULATION = '1'
try {
  postman simulation run postman/simulations/paypal-read/paypal-read.sim.yaml --proxy --lift 'npm --prefix sdk/typescript
test' --output ndjson
} finally {
  Remove-Item Env:RESOLVE_RUN_SDK_EVIDENCE_SIMULATION -ErrorAction SilentlyContinue
}
```

Checkout integration

Source: examples/checkout-store/README.md

Field Brew Supply

Aftercare assurance (SDK 0.7) adds refund tracing, reusable support context and return policy/label workflows. See the workflow guide (../../docs/AFTERCARE_RELIABILITY.md).

The paid receipt now includes **Need help with this order?**: a customer submits an issue, checks its resolution, and opens the private messages/offers/returns portal. The store uses SDK 0.6's `openIssue`, `getOrderResolution` and `createSupportLink` after checking the receipt's HttpOnly session. Generate a support integration key for **field-brew-store** in Operations -> Connections; order-only keys cannot perform support actions. Existing local full-key setup also works for development. See the complete workflow (../../docs/SDK_AFTERCARE.md). Support requests require merchant review and do not authorize a refund. The local Sandbox launcher creates the explicit support grant for its isolated workspace. Its random-port API is separate from the owner's running backend. Portal links require a customer frontend configured for that same API and a matching public origin; otherwise the receipt shows a clear configuration message. The launcher does not start a second frontend automatically.

A small coffee gear shop with six products. It creates PayPal Sandbox orders, captures an approved payment, and sends the store order and items to Resolve through the same Evidence SDK used by an external merchant app. Nothing ships.

Start with your existing Sandbox app

From the Resolve repository root (Node.js 24 and uv required):

```
uv --directory backend sync
npm --prefix sdk/typescript ci
npm --prefix sdk/typescript run build
npm --prefix examples/checkout-store ci
uv --directory backend run python ../examples/checkout-store/scripts/local_sandbox.py
```

Open http://127.0.0.1:4173. The launcher reads only the Sandbox REST app client ID, client secret, and seller merchant ID from `backend/.env` or the corresponding environment variables. It runs a separate Resolve API and a demo merchant in `data/sandbox/`, with Sandbox refund writes disabled and a temporary commerce key. It does not use the database configured for the main Resolve app or print any credentials.

1. Add a product to your cart and select **Continue to PayPal Sandbox**.

2. Sign in to PayPal with a **Personal Sandbox buyer**, then approve the test

purchase. You must approve as the buyer before the store captures.

3. Back in the store, the receipt shows payment status, a Resolve case ID, and

the result of comparing the store's records with PayPal.

4. Select **Check again** to refresh the same Resolve case. If a partial refund

is later recorded outside the store and is visible to the configured Sandbox app, the receipt reports the difference. Rechecking cannot charge or refund.

If checkout succeeds but Resolve is unavailable, the payment stays recorded in the shop. **Check again** retries the SDK sync without submitting the payment again. A pending capture can be checked again without a second capture request.

Connect it to a different Resolve workspace

In this directory, copy `.env.example` to the ignored `.env`. Fill the three `SHOP_PAYPAL_` values and set `RESOLVE_API_URL` to your running FastAPI origin. Issue a commerce key (../../docs/COMMERCE_INTEGRATION.md#start-here) for the merchant whose PayPal Sandbox read connection uses the same seller and app. Set that key as `RESOLVE_API_KEY`. Run `npm start` here.

This mode saves the shop's orders in `data/store.sqlite3` by default. With no credentials, `npm start` serves the catalog and cart preview with checkout disabled. The API binds only to loopback. Use the printed `127.0.0.1` origin, including its port, for the PayPal return flow.

What the demo sends to Resolve

The store calculates totals from its catalog, saves a separate SQLite order, and calls `syncCommerceOrder()` followed by `refreshCommerceOrder()` only after matching a completed capture to the stored order, seller and amount. It sends the app order ID, product lines, currency, PayPal order and capture IDs, the captured amount, and the store's recorded refund total. It never sends buyer login information, a PayPal token, or raw payer details.

The browser starts checkout and displays results; the server holds credentials, validates the approval and capture, and calls the SDK. Receipts are limited to the browser session that created the order. Retries use the saved checkout and capture request IDs and read the provider order before submitting a capture. The demo is Sandbox only.

Store route	Purpose
`GET /api/catalog`	Six products and whether checkout is configured.
`POST /api/orders`	Quote server-side and create or reuse the Sandbox order from `requestId` and product IDs/quantities.
`GET /api/orders/{id}`	Read the current receipt for the owning browser session.
`POST /api/orders/{id}/capture`	Check the returned PayPal `token` and capture only after provider approval, or recover a previous capture.
`POST /api/orders/{id}/verify`	Sync/refresh Resolve evidence for a completed payment without making another capture.

The store follows the PayPal Create order (https://developer.paypal.com/api/orders/v2/orders-create), Capture payment for order (https://developer.paypal.com/api/orders/v2/orders-capture), and Show order details (https://developer.paypal.com/api/orders/v2/orders-get) contracts. PayPal Sandbox buyers and sellers use separate logins; see the existing purchase walkthrough (../../docs/SANDBOX_TEST_PURCHASE.md).

Verify without a real buyer

Install the Chromium build required by the checkout example once:

```
npm --prefix examples/checkout-store exec -- playwright install chromium
npm --prefix examples/checkout-store test
```

If Edge is already installed, you can set `$env:SHOP_TEST_BROWSER_CHANNEL = 'msedge'` for the test instead of downloading a Chromium build.

The default test checks the mobile catalog and cart. For the automated checkout test, run from the repo root:

```
$env:RESOLVE_RUN_CHECKOUT_STORE_SIMULATION = '1'
$env:NODE_USE_ENV_PROXY = '1'
$env:NO_PROXY = '127.0.0.1,localhost'
try {
  postman simulation run postman/simulations/paypal-read/paypal-read.sim.yaml --proxy --lift 'npm --prefix
examples/checkout-store test' --output ndjson
} finally {
  Remove-Item Env:RESOLVE_RUN_CHECKOUT_STORE_SIMULATION,Env:NODE_USE_ENV_PROXY,Env:NO_PROXY -ErrorAction SilentlyContinue
}
```

The checkout test uses Chromium, a real store process, the actual Resolve SDK and API, isolated SQLite databases, and a simulated PayPal approval and capture. It checks that a repeated request produces one capture and the same case, then seeds a simulated external USD 5.00 refund and checks the discrepancy. For an account-backed checkout setup check, pass `--smoke` to the launcher. It creates an unapproved Sandbox order and exits without capturing it.